**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

# Лабораторная работа № 2

## по курсу «Методы оптимизации»

«Метод обратного переменного шага и метод Пауэлла»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

*Москва 2026*

# 1 Задание

1. Самостоятельно изучить методы (https://dzen.ru/video/watch/698cfe82090a5e0bd6f4e33f):
   - обратного переменного шага;
   - Пауэлла.
2. Реализовать оба приведённых выше способа и добавить их в летучку 1; предполагается, что должно быть реализовано все три способа, включая метод Фибоначчи.
3. Результат оптимизации вывести на одном графике: для каждого способа выбирается свой цвет отображения релаксационной последовательности, точки откладываются как на оси $x$, так и на оси $y$. Обязательно указать на графике интервал точности решения.
4. Вывести количество итераций для каждого способа и сделать выводы.
5. Вывести в консоль проверку (лог) на унимодальность.
6. Вывести проверку на минимум по правилу дождя.

**Вариант 4:** функция $f(x) = x^4 - 4x^2$.

# 2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots

function svenn(f, x0, h)
    x1 = x0
    x2 = x0 + h
    if f(x2) > f(x1)
        h = -h
        x2 = x0 + h
    end
    while f(x2) < f(x1)
        x1 = x2
```

```
        x2 = x1 + h
        h = 2 * h
    end
    a = min(x1, x2)
    b = max(x1, x2)
    return a, b
end

function fib(n)
    if n <= 1
        return n
    end
    a, b = 0, 1
    for _ in 2:n
        a, b = b, a + b
    end
    return b
end

function fibonacci_search(f, a, b, eps)
    n = 1
    while fib(n + 2) < (b - a) / eps
        n += 1
    end
    x_seq = Float64[]
    y_seq = Float64[]
    k = 1
    x1 = a + (b - a) * fib(n) / fib(n + 2)
    x2 = a + (b - a) * fib(n + 1) / fib(n + 2)
    f1 = f(x1)
    f2 = f(x2)
    push!(x_seq, x1)
    push!(y_seq, f1)
    push!(x_seq, x2)
    push!(y_seq, f2)
    while k < n
        if f1 > f2
            a = x1
            x1 = x2
            f1 = f2
            x2 = a + (b - a) * fib(n - k + 1) / fib(n - k + 2)
            f2 = f(x2)
        else
            b = x2
            x2 = x1
```

```
            f2 = f1
            x1 = a + (b - a) * fib(n - k) / fib(n - k + 2)
            f1 = f(x1)
        end
        push!(x_seq, f1 < f2 ? x1 : x2)
        push!(y_seq, f1 < f2 ? f1 : f2)
        k += 1
    end
    candidates = [a, b, (a + b) / 2, x1, x2]
    x_min = candidates[1]
    for c in candidates
        if f(c) < f(x_min)
            x_min = c
        end
    end
    push!(x_seq, x_min)
    push!(y_seq, f(x_min))
    return x_min, x_seq, y_seq, n
end

function reverse_variable_step(f, a, b, eps)
    x_seq = Float64[]
    y_seq = Float64[]
    x = a
    h = (b - a) * 0.1
    if h < eps
        h = eps * 2
    end
    push!(x_seq, x)
    push!(y_seq, f(x))
    iters = 0
    while abs(h) > eps
        x_cand = x + h
        if x_cand < a
            x_cand = a
        end
        if x_cand > b
            x_cand = b
        end
        if f(x_cand) < f(x)
            x = x_cand
            push!(x_seq, x)
            push!(y_seq, f(x))
            h = 2 * h
            if x + h > b && h > 0
```

```
                h = (b - x) * 0.5
            end
            if x + h < a && h < 0
                h = (a - x) * 0.5
            end
        else
            h = -h / 2
        end
        iters += 1
    end
    if length(x_seq) == 0 || x_seq[end] != x
        push!(x_seq, x)
        push!(y_seq, f(x))
    end
    return x, x_seq, y_seq, iters
end

function parabola_value(x1, f1, x2, f2, x3, f3, x)
    d12 = (x1 - x2) * (x1 - x3)
    d21 = (x2 - x1) * (x2 - x3)
    d31 = (x3 - x1) * (x3 - x2)
    (abs(d12) < 1e-14 || abs(d21) < 1e-14 || abs(d31) < 1e-14) && return f2
    f1 * (x - x2) * (x - x3) / d12 + f2 * (x - x1) * (x - x3) / d21 + f3 * (x - x1) * (x - x2) / d31
end

function powell_search(f, a, b, eps)
    x_seq = Float64[]
    y_seq = Float64[]
    parabolas = Tuple{Float64,Float64,Float64,Float64,Float64,Float64}[]
    x1 = a
    x3 = b
    x2 = (a + b) / 2
    f1 = f(x1)
    f2 = f(x2)
    f3 = f(x3)
    if f2 > f1 && f2 > f3
        x2 = f1 < f3 ? x1 + (x3 - x1) * 0.382 : x1 + (x3 - x1) * 0.618
        f2 = f(x2)
    end
    push!(x_seq, x1)
    push!(y_seq, f1)
```

```
    push!(x_seq, x2)
    push!(y_seq, f2)
    push!(x_seq, x3)
    push!(y_seq, f3)
    iters = 0
    while (x3 - x1) > eps
        push!(parabolas, (x1, f1, x2, f2, x3, f3))
        denom = (x2 - x3) * f1 + (x3 - x1) * f2 + (x1 - x2) * f3
        if abs(denom) < 1e-14
            break
        end
        x_new = 0.5 * ((x2^2 - x3^2) * f1 + (x3^2 - x1^2) * f2 + (x1^2 - x2^2) * f3) / denom
        if x_new <= x1 || x_new >= x3
            x_new = (x1 + x2) / 2
        end
        f_new = f(x_new)
        push!(x_seq, x_new)
        push!(y_seq, f_new)
        idx_worst = argmax([f1, f2, f3])
        if idx_worst == 1
            x1 = x_new
            f1 = f_new
        elseif idx_worst == 2
            x2 = x_new
            f2 = f_new
        else
            x3 = x_new
            f3 = f_new
        end
        ord = sortperm([x1, x2, x3])
        xs = [x1, x2, x3][ord]
        fs = [f1, f2, f3][ord]
        x1, x2, x3 = xs[1], xs[2], xs[3]
        f1, f2, f3 = fs[1], fs[2], fs[3]
        iters += 1
    end
    x_min = f1 < f2 && f1 < f3 ? x1 : (f2 < f3 ? x2 : x3)
    push!(x_seq, x_min)
    push!(y_seq, f(x_min))
    return x_min, x_seq, y_seq, iters, parabolas
end

function num_derivative(f, x, delta=1e-7)
    return (f(x + delta) - f(x - delta)) / (2 * delta)
```

```
end

function num_second_derivative(f, x, delta=1e-6)
    return (f(x + delta) - 2 * f(x) + f(x - delta)) / (delta^2)
end

function prove_unimodality(f, a, b, n_points=100)
    step = (b - a) / (n_points - 1)
    sign_changes = 0
    prev_der = num_derivative(f, a)
    for i in 2:n_points
        x = a + (i - 1) * step
        der = num_derivative(f, x)
        if (prev_der < 0 && der >= 0) || (prev_der > 0 && der <= 0)
            sign_changes += 1
        end
        prev_der = der
    end
    unimodal = (sign_changes == 1)
    println("                                    [", a, ", ", b, "]: ", unimodal)
    return unimodal
end

function prove_minimum_rule(f, x_min, precision, a=-Inf, b=Inf)
    delta = max(1e-10, precision * 0.5)
    f0 = f(x_min)
    tol = max(1e-9, 1e-6 * max(abs(f0), 1.0))
    check_left = x_min - delta >= a
    check_right = x_min + delta <= b
    rain_left = !check_left || (f(x_min - delta) >= f0 - tol)
    rain_right = !check_right || (f(x_min + delta) >= f0 - tol)
    is_min_rain = rain_left && rain_right
    println("                          :        h=", delta, " : ", rain_left, ", ", rain_right)
    println("                                                  : ", is_min_rain)
    return is_min_rain
end

f(x) = x^4 - 4*x^2
eps = 1e-3

a, b = svenn(f, 0.0, 0.01)
```

```
println("                    :                    [", a, ", ", b, "]")

x_fib, xseq_fib, yseq_fib, iter_fib = fibonacci_search(f, a, b, eps)
x_rvs, xseq_rvs, yseq_rvs, iter_rvs = reverse_variable_step(f, a, b, eps)
x_pow, xseq_pow, yseq_pow, iter_pow, parabolas_pow = powell_search(f, a, b, eps)

prove_unimodality(f, a, b)

x_min = x_fib
println("                                    :")
println("                  : ", iter_fib)
println("                                          : ", iter_rvs)
println("              : ", iter_pow)
println("                                  :")
println("                  : x* = ", x_fib, ", f(x*) = ", f(x_fib))
println("                                          : x* = ", x_rvs, ", f(x*) = ", f(x_rvs))
println("              : x* = ", x_pow, ", f(x*) = ", f(x_pow))

prove_minimum_rule(f, x_min, eps, a, b)

margin = 0.35
min_span_x = (b - a) * 0.06
function local_lims(xseq, yseq)
    xlo, xhi = minimum(xseq), maximum(xseq)
    ylo, yhi = minimum(yseq), maximum(yseq)
    span_x = max(xhi - xlo, min_span_x)
    span_y = max(yhi - ylo, max(abs(ylo), abs(yhi), 1e-8) * 0.08)
    dx = span_x * margin
    dy = span_y * margin
    (xlo - dx, xhi + dx, ylo - dy, yhi + dy)
end
function plot_path_on_curve!(plt, f, xseq, color, marker, label; n_pts=50)
    for i in 1:length(xseq)-1
        x_seg = range(xseq[i], xseq[i+1], length=n_pts)
        plot!(plt, x_seg, f.(x_seg), color=color, lw=1.2, label=(i == 1 ? label : ""))
    end
    plot!(plt, xseq, f.(xseq), color=color, lw=0, marker=marker, ms=6, label="")
```

```
end

p1 = let (xlo, xhi, ylo, yhi) = local_lims(xseq_fib, yseq_fib)
    x_plot1 = range(xlo, xhi, length=500)
    plt = plot(x_plot1, f.(x_plot1), color=:gray, lw=0.5, ls=:dot, label="f(x)", legend=:topright)
    plot_path_on_curve!(plt, f, xseq_fib, :blue, :circle, "")
    vline!(plt, [x_fib - eps, x_fib + eps], color=:black, lw=0.8, ls=:dash, label="  (x)")
    hline!(plt, [f(x_fib) - eps, f(x_fib) + eps], color=:black, lw=0.8, ls=:dot, label="  (y)")
    xlims!(plt, xlo, xhi)
    ylims!(plt, ylo, yhi)
    xlabel!(plt, "x")
    ylabel!(plt, "f(x)")
    title!(plt, "  ,  = $eps")
    plt
end
savefig(p1, "lab2_fib.png")
display(p1)

p2 = let (xlo, xhi, ylo, yhi) = local_lims(xseq_rvs, yseq_rvs)
    x_plot2 = range(xlo, xhi, length=500)
    plt = plot(x_plot2, f.(x_plot2), color=:gray, lw=0.5, ls=:dot, label="f(x)", legend=:topright)
    plot_path_on_curve!(plt, f, xseq_rvs, :red, :square, "")
    vline!(plt, [x_rvs - eps, x_rvs + eps], color=:black, lw=0.8, ls=:dash, label="  (x)")
    hline!(plt, [f(x_rvs) - eps, f(x_rvs) + eps], color=:black, lw=0.8, ls=:dot, label="  (y)")
    xlims!(plt, xlo, xhi)
    ylims!(plt, ylo, yhi)
    xlabel!(plt, "x")
    ylabel!(plt, "f(x)")
    title!(plt, "  ,  = $eps")
    plt
end
savefig(p2, "lab2_rvs.png")
display(p2)

p3 = let (xlo, xhi, ylo, yhi) = local_lims(xseq_pow, yseq_pow)
    x_plot3 = range(xlo, xhi, length=500)
```

```
    plt = plot(x_plot3, f.(x_plot3), color=:gray, lw=0.5, ls=:dot, label="f(x)", legend=:topright)
    for (idx, quad) in enumerate(parabolas_pow)
        x1, f1, x2, f2, x3, f3 = quad
        x_par = range(x1, x3, length=80)
        y_par = [parabola_value(x1, f1, x2, f2, x3, f3, xi) for xi in x_par]
        plot!(plt, x_par, y_par, color=:darkgreen, lw=0.8, ls=:dash, label=(idx == 1 ? "                                    " : ""))
    end
    plot_path_on_curve!(plt, f, xseq_pow, :green, :diamond, "                    ")
    vline!(plt, [x_pow - eps, x_pow + eps], color=:black, lw=0.8, ls=:dash, label="      (x)")
    hline!(plt, [f(x_pow) - eps, f(x_pow) + eps], color=:black, lw=0.8, ls=:dot, label="      (y)")
    xlims!(plt, xlo, xhi)
    ylims!(plt, ylo, yhi)
    xlabel!(plt, "x")
    ylabel!(plt, "f(x)")
    title!(plt, "                                ,     = $eps")
    plt
end
savefig(p3, "lab2_powell.png")
display(p3)
```

Результат представлен на рисунке 1.

```
PS C:\Users\Venya\BMSTUProg\Optimization\lab2> julia main.jl
Метод Свенна: интервал [1.28, 2.56]
Унимодальность на [1.28, 2.56]: true
Количество итераций:
Фибоначчи: 15
Обратный переменный шаг: 13
Пауэлла: 30
Значения минимумов:
Фибоначчи: x* = 1.4138509705698183, f(x*) = -3.999998948487124
Обратный переменный шаг: x* = 1.414, f(x*) = -3.999999635184
Пауэлла: x* = 1.413864265044232, f(x*) = -3.999999024172073
Правило дождя: при h=0.0005 : true, true
Точка является минимумом: true
```

Рис. 1 — Результат

.